Lab 10: Inter-Domain Routing with BGP

Computer Networks · Week 10 · Network Layer / Inter-Domain Routing

.

What this lab does

In this lab you implement a simplified version of the Border Gateway Protocol (BGP), the path-vector protocol that routes between autonomous systems (ASes) on the Internet. Each AS learns routes to a destination prefix from its neighbours, applies its own decision process to pick the best one, and re-advertises that choice, adding itself to the AS-path. You run the protocol to convergence on an AS graph and report the path each AS selects.

BGP routes between networks, and its choice is driven by policy, not distance. The central result of this lab is that an AS will often, and deliberately, choose a longer path because its policy prefers it. That is the defining difference between intra-domain and inter-domain routing.

You will:

- represent an AS-level topology and originate a prefix from one AS;
- implement the BGP decision process: local preference, then AS-path length, then a tie-break;
- implement loop prevention using the AS-path;
- run the protocol to convergence and observe how a single policy change reroutes traffic, including onto a longer path;
- read the AS-path out of a real BGP UPDATE in Wireshark and reconcile it with your simulator;
- run real BGP routers (FRRouting containers) and watch the AS-path grow hop by hop.

.

Setup

Three tools, one per layer of the lab. For Parts A to D (the simulator) use any C++ compiler or Python 3, and implement the protocol yourself without a routing library. For Part E you need Wireshark and the provided capture `lab9_bgp_session.pcap`. For Part F you need Docker and the provided `lab9_frr` folder (compose file and router configs); these run against the standardized lab image, so no image download is required on the day.

Conventions and the decision process (the shared specification)

- An autonomous system is identified by a number. A peering is an undirected link between two ASes that exchange routes.

- One AS originates the destination prefix. Its AS-path to the prefix is empty, because it is the origin.
- A route is an AS-path (the list of ASes to traverse) and a next-hop AS (the neighbour the route was learned from).
- When an AS advertises its best route to a neighbour, the neighbour forms a candidate route by prepending the advertiser's AS number to the advertised AS-path. The advertiser's AS number becomes the next hop.
- **Loop prevention.** An AS rejects any candidate route whose AS-path already contains its own AS number. This is how BGP prevents loops, and it is why BGP has no count-to-infinity problem.
- **The decision process.** Among the candidate routes it accepts, an AS selects the best one by applying these rules in order until one route wins:
    1. highest local preference (a per-AS policy value; the default is 100);
    2. shortest AS-path;
    3. lowest next-hop AS number (a deterministic tie-break standing in for lowest router id).

Local preference is set per AS, as a value attached to routes learned from a particular neighbour. It is listed per scenario below.

The topology (used by every scenario)

Six autonomous systems. The destination prefix is originated by AS1. Peerings:

AS	Peers with
AS1	AS2, AS3
AS2	AS1, AS3, AS4
AS3	AS1, AS2, AS5
AS4	AS2, AS5, AS6
AS5	AS3, AS4, AS6
AS6	AS4, AS5

Test vectors (your converged output must match these exactly)

Each entry is the AS-path an AS installs to reach the prefix originated by AS1, and its next-hop AS.

Scenario 1, no policy (every local preference is the default 100).

AS	AS-path	Next hop
AS2	AS1	AS1
AS3	AS1	AS1
AS4	AS2, AS1	AS2
AS5	AS3, AS1	AS3

AS6	AS4, AS2, AS1	AS4

AS6 has two equal-length candidates, via AS4 (AS-path AS4, AS2, AS1) and via AS5 (AS-path AS5, AS3, AS1). The AS-path lengths tie at 3, so the tie-break picks the lower next-hop AS, which is AS4.

Scenario 2, AS6 sets local preference 200 on routes learned from AS5.

Only AS6 changes. It now installs AS-path AS5, AS3, AS1 with next hop AS5. The two candidates are still the same length, but local preference is applied before AS-path length, so AS6's policy flips its choice to the route via AS5.

Scenario 3, AS5 sets local preference 50 on routes learned from AS3.

AS	AS-path	Next hop
AS5	AS4, AS2, AS1	AS4
AS6	AS4, AS2, AS1	AS4

AS5's shortest route is via AS3 (AS-path AS3, AS1, length 2). By lowering the local preference of routes from AS3 below the default, AS5 makes its route via AS4 (AS-path AS4, AS2, AS1, length 3) win instead, because local preference is decisive before AS-path length. AS5 has deliberately chosen a longer path. AS6 follows, because its route now depends on AS5's choice.

.

Part A: Topology and origination

What you do: build the AS graph and originate the prefix.

Your task:

1. Store the peerings as an adjacency structure. Each peering appears in both ASes' neighbour lists.
2. Mark AS1 as the origin of the prefix, with an empty AS-path.
3. As a check, print each AS and its peers and confirm the graph matches the topology table.

Part B: The BGP decision process

What you do: implement route selection at a single AS, given the routes its neighbours currently advertise.

Background: an AS forms one candidate route per neighbour by prepending the neighbour to that neighbour's advertised AS-path. It discards any candidate whose AS-path already contains itself. Among the rest it applies local preference, then AS-path length, then the lowest next-hop AS number, in that order.

Your task:

1. Implement the loop check: reject a candidate whose AS-path contains this AS.
2. Implement the decision process in the specified order and return the single best route.
3. Test it on AS6 in Scenario 1, where two equal-length candidates force the tie-break, and confirm it selects the route via AS4.

Part C: Convergence

What you do: run the protocol until no AS changes its best route, then report the result.

Background: BGP converges by repeated advertisement. In each round every AS advertises its current best route to its neighbours, and every AS re-runs its decision process on what it has heard. Repeat until a full round produces no change. Use synchronous rounds, where every AS decides from the routes advertised at the start of the round.

Your task:

1. Run the protocol to convergence on the topology with no policy set.
2. Report each AS's installed AS-path and next hop, and confirm they match the Scenario 1 vector.

Part D: Policy and analysis

What you do: apply the two policy scenarios, observe the rerouting, and account for it.

Your task:

1. Apply Scenario 2 (AS6 prefers routes from AS5) and confirm only AS6 changes, to the route via AS5. Explain why local preference overrides the AS-path tie-break.
2. Apply Scenario 3 (AS5 de-prefers routes from AS3) and confirm AS5 installs the length-3 path via AS4 instead of the length-2 path via AS3. State plainly that AS5 has chosen a longer path, and explain why this cannot happen in the Dijkstra sink-tree lab but is normal in BGP.
3. Explain how the AS-path prevents routing loops, and why this means BGP does not suffer the count-to-infinity problem that a distance-vector protocol would on the same topology.
4. In two or three sentences, explain why inter-domain routing uses policy-based path selection at all, rather than simply minimising the number of AS hops.

Part E: Read a real BGP session in Wireshark

What you do: open the provided capture of a real BGP session and read the UPDATE message, then reconcile its AS-path with the one your simulator computed.

Background: BGP is not only a decision algorithm, it is a protocol that runs over a TCP connection on port 179. Two speakers first exchange OPEN messages to establish the session, send periodic KEEPALIVE messages, and advertise routes in UPDATE messages. An UPDATE carries the prefix being advertised and a set of path attributes, one of which is the

AS_PATH: the very list your simulator builds. This part shows you that the AS-path you treated as a list in code is a real field in a real packet.

The capture `lab9_bgp_session.pcap` is a session between AS4 (198.51.100.4) and AS6 (198.51.100.6) in which AS4 advertises the prefix 203.0.113.0/24, originated by AS1, to AS6.

Your task:

1. Open the capture and identify the three BGP message types present: OPEN, KEEPALIVE, and UPDATE. Note that they ride on TCP port 179.
2. Select the UPDATE message and expand its path attributes. Read the AS_PATH attribute and the advertised prefix (the NLRI).
3. Confirm the AS_PATH is the sequence 4, 2, 1, and the prefix is 203.0.113.0/24.
4. Reconcile this with your Part C output. The AS_PATH 4, 2, 1 in this real UPDATE is exactly the route your simulator installed at AS6 in Scenario 1 (AS6 reaches AS1 via AS4, with AS-path AS4, AS2, AS1). State in one sentence how the attribute in the packet corresponds to the list in your code.

Part F: BGP on real routers (FRRouting containers)

What you do: bring up a small topology of real BGP routers, observe the route one router learns, capture the live UPDATE that carried it, and change a policy to see the route move.

Background: a simulator and a capture show the logic and the packet. Real routing software shows both at once, running the actual protocol. FRRouting (FRR) is a free, open-source routing suite that speaks real BGP. You will run three FRR routers as containers, each a separate AS, and watch AS1's prefix propagate with the AS-path growing by one AS at each hop.

The provided topology is a line of three routers: r1 (AS65001) originates 203.0.113.0/24, r2 (AS65002) is transit, and r3 (AS65003) is the receiver. Private AS numbers are used.

Your task:

1. Bring up the topology with `docker compose up -d` using the provided compose file and router configs.
2. On r3, run `vtysh -c "show ip bgp 203.0.113.0/24"` and read the AS-path of the learned route. Confirm it is 65002, 65001, which shows each transit AS prepending itself, exactly the mechanism from Part B.
3. Capture the BGP session on the r2-to-r3 link with tcpdump, filtering on TCP port 179, and open the capture in Wireshark. Find the UPDATE and confirm its AS_PATH matches what `show ip bgp` reported.
4. Apply one policy change: on r3, lower the local preference of the route, or on r2, prepend its own AS an extra time. Re-run `show ip bgp`, capture the new UPDATE, and describe what changed and why, connecting it to your Part D policy results.

Submit the output of `show ip bgp` before and after your policy change, and your captured UPDATE.

.

What to submit

- Your source file (C++ or Python) implementing the topology, the decision process, loop prevention, and convergence.
- Program output reproducing the Scenario 1, 2, and 3 vectors exactly.
- The written answers to Part D.
- Part E: your reading of the UPDATE's AS_PATH and prefix, and the one-sentence reconciliation with your simulator.
- Part F: the `show ip bgp` output before and after your policy change, and the captured UPDATE.

Appendix: the interface each program must implement

- **Decision at one AS:** `best_route(as, advertised_routes, localpref) -> (as_path, next_hop)`, applying loop prevention then the three-rule decision process.
- **Convergence:** `run_bgp(peerings, origin, localpref) -> {as: (as_path, next_hop)}`, running synchronous rounds until no best route changes.

Only the converged AS-paths and next hops are graded, not the types. A candidate AS-path is formed by prepending the advertiser; a route is rejected if its AS-path already contains the deciding AS.